



SOUVERAINETÉ NUMÉRIQUE

2025



HORS SÉRIE

Souveraineté numérique.

Comprendre et pivoter
pour assurer l'autonomie
stratégique.

THERE IS A BETTER WAY



| SOMMAIRE |

01

DÉPENDANCES
C'est affaire de dépendances

05

CHANGER DE CULTURE
Changement de culture

02

STANDARDS
Des standards

06

DÉPLACER LE FINANCEMENT

03

COMMUNS NUMÉRIQUES
Logiciel libre, open source, commun numérique

07

MISE EN PRATIQUE

04

FAIRE COMMUN
Faire commun



| LES AUTEURS |

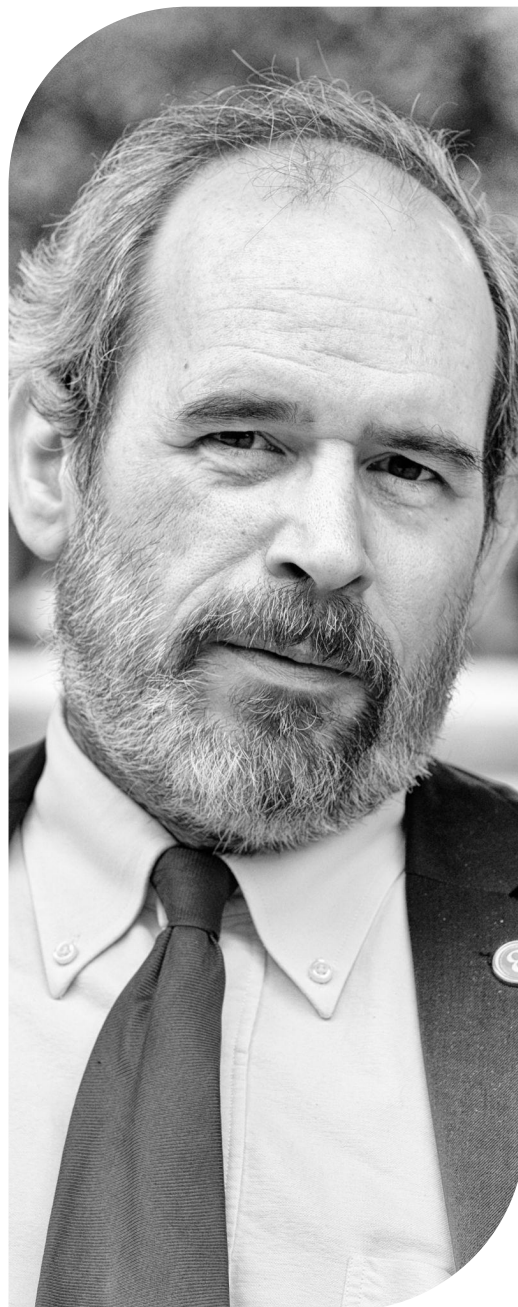
Benjamin Bayart

OCTO Technology

Senior manager

En charge du pôle
Cloud Souverain

Par sa pratique technique de **développeur**, **d'administrateur système**, d'opérateur d'infrastructures **réseau**, ainsi que par ses connaissances juridiques sur les données personnelles et sa pratique en amateur éclairé¹ du **droit administratif**, il apporte un éclairage particulier, mêlant technique, droit, et organisation managériale, à la question de la souveraineté numérique, lue comme une volonté de résilience et **d'autonomie stratégique**.



¹ On a parlé, autrefois, d'exégètes amateurs...



Ludovic Chauvaux

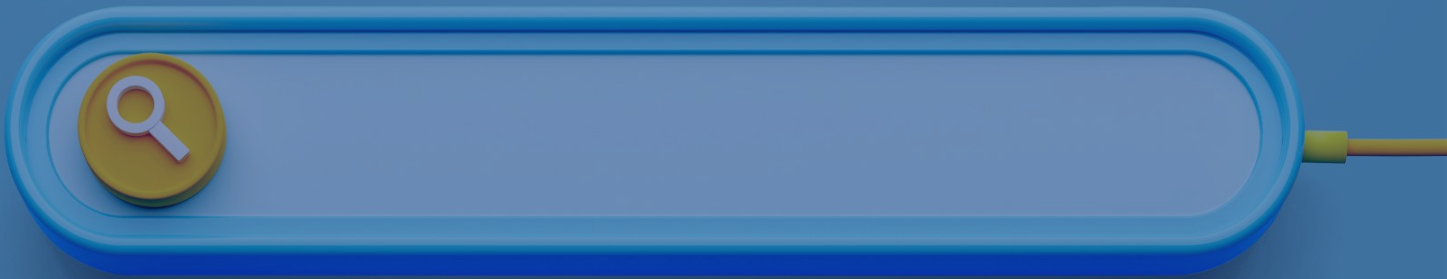
OCTO Technology

Senior manager

En charge de l'équipe
Architecture
& Tech Advisory.

Avec une carrière marquée par des rôles de leadership tels Lead Architect ou CTO, il a su conjuguer **expertise technique et vision stratégique**. Passionné par l'infrastructure et la gestion des systèmes d'information, ses nombreuses missions l'ont amené à développer une **pensée systémique** prêt à étendre ses compétences techniques et de leadership, ce qui lui a permis de **relier efficacement l'opérationnel et la stratégie de l'entreprise**.





INTRODUCTION



| INTRODUCTION |

Ce manifeste présente une analyse approfondie de ce qu'est la question de l'autonomie stratégique, et l'objectif de résilience dans le numérique.

Notre sujet n'est pas d'avoir une souveraineté nationaliste, mais d'avoir une gestion raisonnée des dépendances techniques, pour viser la résilience et l'autonomie. Et cette pensée se fait sur des lignes de code, sur des protocoles réseau et sur des choix technologiques.

Nous explorons ici une réalité trop négligée : notre dépendance croissante à des infrastructures numériques dominées par des acteurs hégémoniques, et les risques stratégiques, économiques et politiques qui en découlent. Par une analyse rigoureuse, mais accessible, nous essayons de comprendre les mécanismes de cette dépendance — du cloud aux logiciels métiers — et de proposer des stratégies concrètes pour gagner en autonomie numérique.

Nous préconisons une approche lucide et constructive : investir dans les communs numériques, adopter des standards ouverts, et surtout, changer de culture pour passer d'une logique de consommation passive à celle d'une contribution active.

La résilience numérique ne se décrète pas, elle se construit, ensemble, patiemment, en assumant nos choix technologiques et en soutenant les outils qui garantissent notre autonomie d'action.

Quand on est contraint par une dépendance, on n'est plus libre de choisir, et c'est un enjeu crucial.

Ce manifeste est une invitation à penser autrement notre rapport au numérique. Il s'adresse à tous ceux qui, au sein des entreprises, des administrations ou de la société civile, veulent bâtir un futur numérique plus résilient, et donc plus libre.



DÉPENDANCES

01

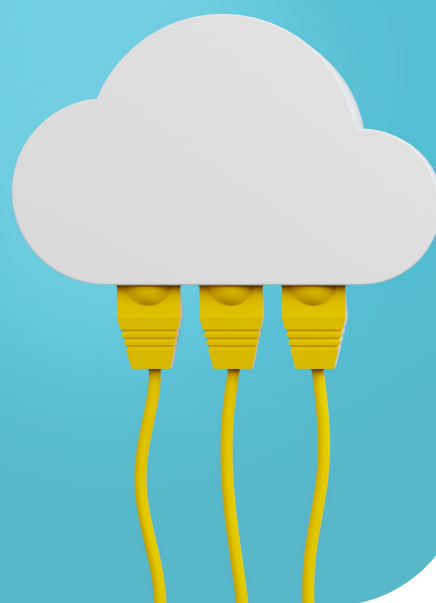
C'est une affaire de dépendances

Le sujet du cloud souverain est revenu dans le débat public il y a 5 ans, lorsque la Cour de Justice de l'Union Européenne (CJUE) a rendu son arrêt Schrems II. Pour la deuxième fois en quelques années, elle annulait un acte de l'Union Européenne et affirmait que le droit étasunien n'était pas assez protecteur des données personnelles par rapport aux normes européennes.

Juridiquement, la décision est intéressante à de nombreux égards. Mais une lecture plus géopolitique est intéressante également : on peut limiter le libre échange en appliquant le droit des personnes (le RGPD et la protection des données personnelles en Europe, c'est du droit des personnes, pas du droit des affaires). Le RGPD peut alors être considéré comme un outil de protectionnisme réglementaire.

Si on fait une lecture stricte de l'arrêt de la CJUE, partout où le droit étasunien s'applique sur les données personnelles, le droit européen est violé. C'est par exemple le cas quand un système d'information est hébergé par les grands fournisseurs de cloud (Amazon, Microsoft, Google), même quand ces systèmes sont hébergés sur des serveurs en Europe. D'une part, parce que ces fournisseurs ne peuvent pas garantir que des données ne seront à aucun moment déplacées vers les USA, et d'autre part parce que le droit étasunien s'applique à ces entreprises où qu'elles opèrent dans le monde.

Le RGPD peut alors être considéré comme un outil de protectionnisme réglementaire.





Les arrêts Schems I & II de la CJUE

La CJUE est la Cour de justice de l'Union Européenne qui peut être saisie par toutes les cours suprêmes de tous les États Membres (pour la France, par la Cour de Cassation, par le Conseil d'État ou par le Conseil Constitutionnel). La CJUE est saisie par une Cour suprême quand une question se pose sur la façon d'appliquer le droit de l'Union, et que cette question présente un caractère nouveau qui fait qu'il n'est pas possible d'y répondre directement, pour assurer que le droit de l'Union est interprété et appliqué de la même façon dans tous les États Membres.

Dans une affaire qui, au départ, oppose M. Maximilien Schrems et Facebook, la CJUE a été saisie par la cour suprême d'Irlande, pour savoir si la décision de la CNIL local disant que les transferts de données par Facebook vers les USA étaient bien conformes au droit de l'Union.

Le contexte historique est à noter : le premier arrêt Schrems est rendu alors qu'on sort tout juste des révélations d'E. Snowden sur l'étendue de la surveillance exercée par les services Américains sur l'ensemble du monde.

La CJUE va apporter une solution dans l'arrêt dit "Schrems I", d'octobre 2015 :

- l'accord international, le Safe Harbor Act, qui autorise les échanges de données personnelles entre l'UE et les USA est invalidé ;
- les garanties apportées par le droit des USA ne sont pas suffisantes pour être compatibles avec le droit UE.

Six mois après cette décision, un second accord est signé entre les USA et l'UE, le "Privacy Shield", organisant la libre circulation des données personnelles entre les USA et l'UE.

Dans le cadre de la même affaire, le sujet sera de nouveau porté devant la CJUE (de savoir si les transferts fait par Facebook sont couverts par le Privacy Shield).

Dans cet arrêt Schrems II, en octobre 2020, la CJUE tranchera le litige une deuxième fois, dans le même sens : incompatibilité du droit des USA avec la protection des données personnelles garantie par la Charte des droits fondamentaux de l'UE.

Un troisième accord, dit "Data protection framework" entre en vigueur en juillet 2023.

Certaines des structures prévues par cet accord sont défaites dans le cadre des Executive Order pris par D. Trump au début de son second mandat (janvier 2025).

La CJUE va apporter une solution dans l'arrêt dit "Schrems I", d'octobre 2015 :

- l'accord international, le Safe Harbor Act, qui autorise les échanges de données personnelles entre l'UE et les USA est invalidé ;
- les garanties apportées par le droit des USA ne sont pas suffisantes pour être compatibles avec le droit UE.





Du droit de portée extra-territoriale

Pour ce qui nous intéresse ici, à savoir le régime de protection juridique des données personnelles, autant l'Union Européenne que les États-Unis d'Amérique ont un droit qui a une portée au-delà de leur territoire. Mais la méthode diffère grandement.

Le droit des USA est souvent de portée globale, sur le principe, les Cours de justice des USA considérant qu'il s'applique partout dans le monde. C'est par exemple le cas d'un certain nombre de textes qui encadrent le monde des affaires (p. ex. les textes anti-corrupcions) dont les USA considèrent qu'ils s'appliquent à toute transaction commerciale réalisée en dollars américains.

Les textes qui permettent aux agences étasuniennes d'accéder aux données personnelles sont de portée explicitement extra-territoriale. C'est le cas d'au moins deux familles de textes, le **Cloud Act** d'une part (qui permet, avec le mandat d'un juge, d'accéder à toutes données, où qu'elles soient dans le monde) et les dérivés du Patriot Act, par exemple les textes de la **FISA 702** (qui permet à différentes agences de renseignement, éventuellement après saisine d'une autorité ad-hoc, d'accéder à des données).

Du côté de l'Union Européenne, la protection des données personnelles est rattachée, dans les jugements de la CJUE, à la Charte des droits fondamentaux de l'Union (adossée aux traités depuis le traité de Nice, décembre 2000). La protection des données personnelles est considérée comme une conséquence du droit à une vie privée et familiale, un droit attaché à l'individu.

Le RGPD par exemple, protège les données personnelles des Européens, et ses règles s'appliquent à toutes les organisations qui sont amenées à traiter ces données, où qu'elles soient dans le monde. Et donc, de fait, les textes européens qui traitent des données personnelles s'appliquent, y compris aux traitements de ces données qui ont lieu en dehors du territoire de l'Union.

Les textes qui permettent aux agences étasuniennes d'accéder aux données personnelles sont de portée explicitement extra-territoriale.





C'est pourquoi on a vu resurgir, à la suite de cet arrêt, la notion de cloud souverain : un fournisseur de cloud qui serait soumis au droit de l'Union Européenne et qui ne serait pas soumis au droit des États-Unis.

En pratique, cette lecture est simpliste. La protection des données personnelles n'est qu'une des nombreuses contraintes qui s'imposent à un système d'information. On retrouvera ici tout le panel habituel de l'industrie (sécurité, loi applicable, résilience, coûts, secret défense, etc).

On peut par exemple prendre l'analyse d'un système informatique régalien. Mettons, le fichier de l'état civil. Si le pays qui souhaite numériser son état civil veut qu'il soit résilient, y compris quand il y a des soubresauts politiques majeurs sur la scène internationale, il va imposer des contraintes sur le mode d'hébergement. On retrouvera des règles de sécurité (il ne faut pas que le système puisse être accédé indûment) et des règles de souveraineté (si une puissance étrangère peut éteindre le système, ça ne va pas). Ici, on trouve un lien direct entre régalien (qui relève de l'État) et souverain. Dans cette approche régaliennne, la notion de "pays allié" est une notion forte, de même que de décider à quel point un pays accepte de dépendre de ses alliés (par exemple, la France a toujours considéré que pour sa souveraineté industrielle et militaire, elle devait produire ses propres avions de chasse, au lieu de les acheter à son allié américain comme le font les allemands).

Mais on peut faire la même analyse dans le monde de l'entreprise. Une entreprise qui s'apprête à héberger un de ses systèmes sur du cloud doit se poser la question de la dépendance technique à son fournisseur. D'une part, sur le plan financier : si mon fournisseur décide de tripler ses tarifs, suis-je prisonnier, et en combien de temps puis-je changer de fournisseur ? D'autre part sur le plan opérationnel : si le fournisseur coupe le service, est-ce que l'entreprise survit ? Si la réponse est non, et si on en a les moyens, il faut étudier la question.

Si mon fournisseur décide de tripler ses tarifs, suis-je prisonnier, et en combien de temps puis-je changer de fournisseur ?

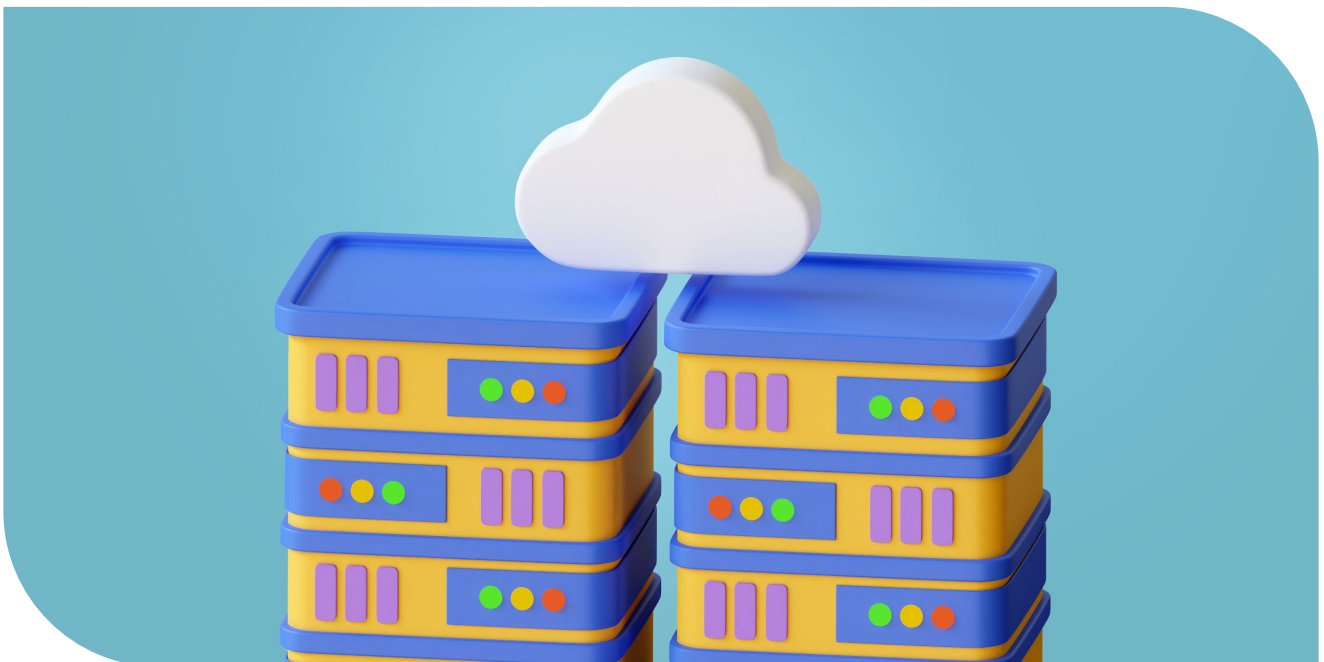
Si le fournisseur coupe le service, est-ce que l'entreprise survit ?



C'est une erreur d'analyse qu'ont fait beaucoup d'entreprises russes, et qui a été mise en évidence par la guerre en Ukraine. Les sanctions économiques contre la Russie, au déclenchement de la guerre, ont entraîné la fermeture des services de cloud de grands fournisseurs (AWS, par exemple) pour beaucoup d'entreprises russes, qui se sont retrouvées du jour au lendemain privées de leurs systèmes informatiques. Cela montre deux choses : d'une part, que ces entreprises n'ont pas tenu compte de leur dépendance technique à leur fournisseur, d'autre part, que les États-Unis sont susceptibles de couper ces services pour des raisons politiques.

Ce travail de recherche des dépendances techniques et économiques, pour contrôler les risques, n'est pas nouveau, y compris dans le domaine informatique.

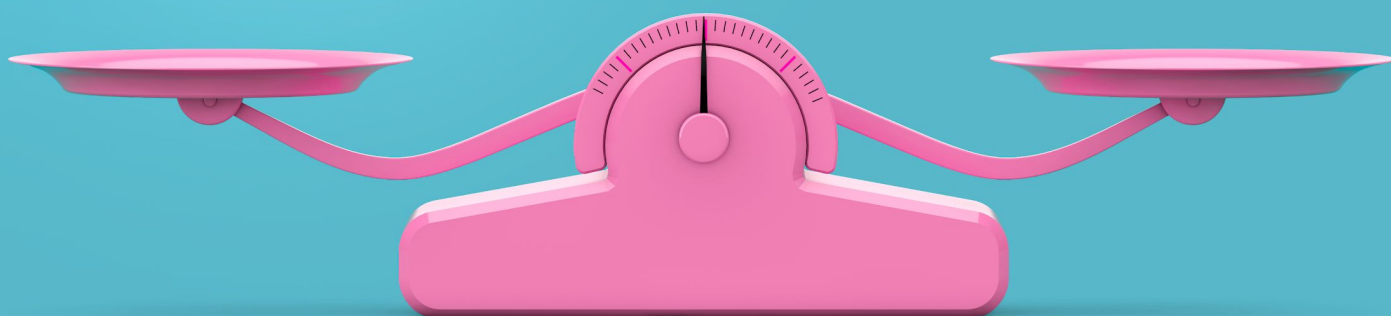
Il y a quelques années, le modèle de prix d'Oracle est devenu trop compliqué à maîtriser pour beaucoup de services informatiques. Là où, il y a 20-30 ans, le réflexe était "si j'ai besoin d'une base de données, je vais mettre Oracle, j'ai une licence, j'ai l'habitude, tout va bien", il est progressivement devenu "si je peux éviter Oracle, je vais éviter, c'est trop cher". Et ce "tout sauf Oracle" a beaucoup porté le déploiement de PostgreSQL à grande échelle. On retrouve ces dernières années le même sujet autour des licences VMWare. Le modèle tarifaire est subitement parti en vrille, suite au rachat de l'entreprise. Et beaucoup de DSI utilisatrices du produit sont donc en train d'étudier des pistes pour en sortir.





STANDARDS

02



Des standards

Gérer la dépendance, qu'elle soit technique ou vis-à-vis d'un fournisseur, c'est soit l'accepter, soit pouvoir changer de solution, idéalement en ayant en permanence plusieurs solutions mises en œuvre. Par exemple, comme le font les industriels pour leurs pièces détachées, en ayant continuellement plusieurs fournisseurs. Le sujet des standards devient alors un point central. Si vous avez besoin de boulons, et donc que vous voulez avoir trois fournisseurs de boulons référencés, vous avez implicitement besoin d'un standard sur les boulons. Si les diamètres et les pas de vis ne sont pas standardisés, ou pire si un élément mécanique (la forme du tournevis, par exemple) est encore soumis à un brevet, il devient beaucoup plus délicat d'avoir plusieurs fournisseurs.

Dans le domaine informatique, on trouve ça de manière régulière, sans y prêter attention puisque ce sont des usages qui se passent bien. Par exemple, un langage de programmation comme le C est raisonnablement standardisé, il existe des dizaines de compilateurs, des centaines peut être, édités par toutes sortes de structures, publiques, privées, communautaires, disponibles sous des contrats de licence très variés. Le passage d'un compilateur à l'autre n'est en pratique pas trivial, mais reste très réaliste. Et donc, si le contrat de licence du compilateur que vous utilisez habituellement n'est plus acceptable, changer de compilateur n'est pas hors de portée.

On trouve des effets similaires quand des services sont bien normalisés, par exemple par les protocoles réseau. Il y a des dizaines de serveurs DNS disponibles sur le marché, avec là aussi toutes sortes de contrats de licence, ou différents serveurs web. Le point essentiel ici est l'existence d'un standard, ouvert, connu, librement implémentable par n'importe quel éditeur de logiciels. En pratique, on peut considérer que sitôt qu'il existe plusieurs implémentations, très compatibles entre elles, on a affaire à un standard, même si formellement ce n'est qu'un standard de fait.

Tous les exemples que nous citons où il y a des frottements, ce sont des services pour lesquels il n'est pas défini de standard, et donc où les services concurrents, quand ils existent, ne sont pas substituables. Si on prend l'exemple d'Oracle, il existe des produits concurrents (les moteurs de base de données, ça ne manque pas), mais ils sont tous plus ou moins incompatibles entre eux, la norme SQL n'étant pas assez stricte pour garantir la substituabilité des produits.

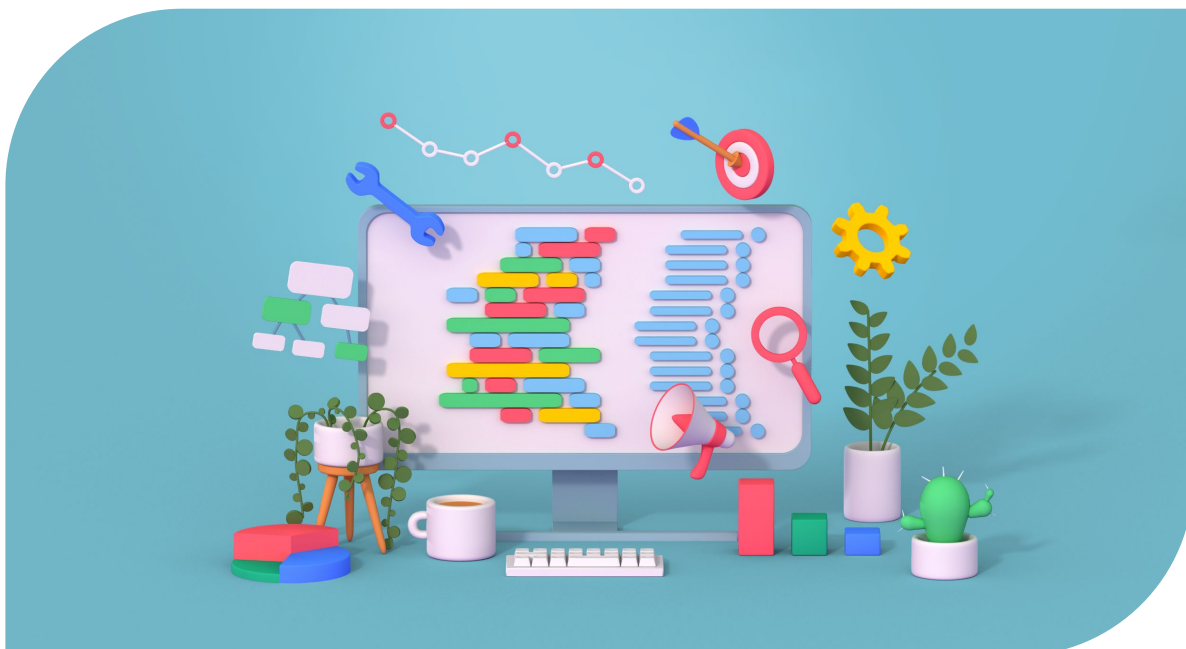
On voit donc ici remonter un problème ancien : l'informatique a besoin de normes, publiées, ouvertes, qui puissent être librement implémentées, et suffisamment précises pour que les produits qui s'appuient sur ces normes soient raisonnablement substituables. L'effort de normalisation, et le respect des normes, n'est donc pas du tout un vieux combat des années 70, comme on le croit parfois, mais bien un besoin très actuel et très moderne. Participer à des groupes internationaux de normalisation n'est donc pas un luxe inutile pour occuper des universitaires poussiéreux et désœuvrés, mais faire une œuvre industrielle utile à tous.

Mais bien entendu, l'existence de standards n'est pas suffisante pour résoudre tous les problèmes. Il y a deux limitations évidentes. La première est qu'en général on standardise le service rendu (par exemple, la notion de serveur DNS, que n'importe quel client peut interroger selon toujours la même norme), mais pas la façon dont ce service vient s'intégrer dans un système d'information complet. Ainsi, la syntaxe des fichiers de configuration, les outils de provisioning, les interfaces de pilotage, de supervision, de détection d'incidents, ne sont en général pas standardisés. Or, intégrer un outil donné dans la complexité d'un système d'information, c'est complexe, coûteux et contraignant.

On peut effectivement remplacer le serveur DNS Bind, fourni par l'ISC par, par exemple, Knot, fourni par CZ.NIC. D'un point de vue client, le service rendu sera le même, peu de reconfiguration à faire. Mais toute l'intégration dans le système d'information sera à revoir, les API pour piloter les deux systèmes se ressemblent peu, les fichiers de configuration diffèrent. Le service principal (implémenter le protocole DNS) est le même, mais tous les services annexes vont différer.

La deuxième limitation vient de l'absence d'implémentation d'un standard. C'est par exemple le cas pour beaucoup de langages de programmation modernes, pour lesquels le langage est documenté convenablement, mais pour lesquels il n'existe qu'une implémentation du compilateur ou de l'interpréteur. C'est le cas par exemple pour Go (dans les langages compilables) ou pour Python, Php, ou Perl (pour les langages interprétés). Ces langages ne sont pas des implémentations d'un standard contraignant, ils forment plutôt une spécification ouverte d'un langage qui n'est pas normalisé. Cette position leur permet d'évoluer (chaque nouvelle version de ces langages apporte de nouvelles fonctionnalités, de nouvelles optimisations), ce qu'un standard permet moins. Pour faire bouger le langage C, ou le langage COBOL, il faut écrire une nouvelle version de la norme, et espérer que tous les éditeurs de toutes les implémentations vont suivre cette nouvelle norme de manière raisonnable. C'est plus stable pour l'industrie, mais ça ralentit l'innovation.

Dans ce type de contexte, la création d'un standard contraignant n'est pas forcément la bonne réponse, puisque le standard se manifestera par un ralentissement des innovations et donc un risque élevé de n'être pas adopté. Dans ce contexte-là, la réduction du risque se fait par le contrat de licence.





03

COMMUNS NUMÉRIQUES

Logiciel libre, open source, commun numérique

On vient de le voir : quand l'industrie dispose d'un standard, efficace et communément repris, le problème de dépendance n'existe pas. À l'opposé, quand on utilise un système entièrement fermé, le problème de dépendance finira forcément par arriver et il n'y a rien à faire contre ça, il faut juste sortir de ces systèmes si on veut retirer la dépendance. Entre les deux, on trouve tous ces systèmes qui forment une spécification ouverte d'un service en perpétuelle évolution, que ce soient des langages (Go, Python, etc.), des services comme ceux d'un cache clé-valeur (Redis), d'un orchestrateur (Kubernetes) d'un bus de message (Kafka), etc.

Ici, pour les besoins de notre démonstration, nous devons distinguer deux notions. Nous allons donc (un peu arbitrairement) séparer la notion de **logiciel libre** de la notion de **logiciel open source**. Le point commun entre les deux peut être un contrat de licence dit ouvert (ou libre) comme la licence GPL, la licence MIT ou la licence Apache. La différence de principe entre les deux, c'est la gouvernance du projet. Pour notre propos, nous appellerons **logiciel open source** un logiciel écrit et maintenu par un organisme, qui a publié le code source, mais qui ne permet pas (ou qui ne simplifie pas) la construction à partir du code source, qui ne permet pas (ou peu) à des développeurs de "rentrer dans le code" pour le faire évoluer, qui n'accepte jamais ou presque jamais les évolutions du code venues de l'extérieur. Le contrat de licence peut changer à tout moment. La garantie apportée contre les dangers de la dépendance est donc faible.



Nous appellerons alors *logiciel libre* ce qui se rattache à la définition traditionnellement défendue par la Free Software Foundation : librement utilisable, libre accès au code, librement modifiable, et diffusion libre des versions modifiées ; en y ajoutant cette importante nuance : qu'il existe un développement communautaire fort de l'outil. L'existence d'un "point central" n'est pas en soi un obstacle, mais ce point central ne doit pas être hégémonique. On peut donc retenir quelques critères simples :

- il existe un développement communautaire (des contributions au code, significatives, viennent de la communauté plutôt que du "point central") ;
- il existe une communauté d'utilisateurs ;
- il arrive que les modifications proposées par la communauté soient imposées par la communauté à la maison mère.

Les précédents sont nombreux où le recours au logiciel libre a permis de traiter le problème. C'est par exemple ce qui s'est produit quand le "tout sauf Oracle" a propulsé PostgreSQL sur le devant de la scène. Le développement du logiciel est communautaire, sa licence est libre, bref, il coche tous les critères qui permettent de protéger les organisations qui l'utilisent.

La nuance qui est faite ici entre open source et logiciel libre, qui pourrait sembler intransigeante, est celle qui permet d'analyser les effets du changement de licence de *Terraform*. Quand *Hashicorp* a décidé de modifier son contrat de licence, beaucoup d'usages de *Terraform* sont devenus problématiques, voire illégaux. Or ce logiciel avait cristallisé une montagne de développements communautaires. Beaucoup d'acteurs de l'hébergement, du cloud, fournissent des modules permettant à *Terraform* de piloter les infrastructures mises à disposition des clients. De fait, en refermant les usages possibles du produit, l'éditeur a compliqué la vie des utilisateurs, mais il a aussi capturé pour son bénéfice privé l'effort de développement des autres contributeurs.

- Il existe un développement communautaire (des contributions au code, significatives, viennent de la communauté plutôt que du "point central") ;

- Il existe une communauté d'utilisateurs ;

- Il arrive que les modifications proposées par la communauté soient imposées par la communauté à la maison mère.

La réponse a été apportée par **OpenTofu** (un outil de provisionning d'infrastructure issu de **Terraform**), en basculant sur le modèle du logiciel libre : repartir de la dernière version du code qui soit légalement disponible, et reprendre le développement à partir de là d'une version communautaire, maintenue par des développeurs payés par des utilisateurs ou par des contributeurs.

En pratique, des logiciels libres qui ont comme effet de réduire le danger de la dépendance, il y en a beaucoup, et si on observe le phénomène sur les dernières décennies, le logiciel libre s'est très largement imposé. C'est par exemple le cas du noyau **Linux**, qui occupe une part de marché colossale. On le retrouve dans les téléphones (**Android**), dans l'embarqué (des box, des décodeurs télé, des télé connectées, etc), dans le cloud (la majorité des VM déployées dans le cloud), etc.

Si on regarde les caractéristiques essentielles de ces logiciels, on constate que la licence libre est nécessaire, mais pas suffisante. Ceux qui permettent d'apporter de l'indépendance technologique, de la souveraineté, peuvent être décrits comme des communs numériques essentiels.



**On constate que
la licence libre est
nécessaire, mais
pas suffisante.**



04

FAIRE COMMUN

Faire commun

On l'a vu, le remplacement d'une solution logicielle propriétaire qui pose un problème de souveraineté (dépendance technologique, risque financier, risque stratégique, etc) par un logiciel libre est un projet en soi. Souvent ambitieux et coûteux, parce que ces systèmes ne sont pas des standards, mais aussi parce que même quand ils sont standards, la refonte du système d'information qui en découle est complexe, il faut reprendre l'intégration de l'ensemble des systèmes.

Si on veut faire un parallèle avec l'aménagement d'un logement, remplacer un outil logiciel qui respecte un standard, c'est équivalent à repeindre une pièce, assez léger. Remplacer un outil par un autre qui n'a pas le même mode d'intégration, c'est plutôt comme transformer une chambre en salle de bain ou en chaufferie, il va y avoir des impacts partout, toute la tuyauterie à revoir et il est probable que la maison soit inhabitable pendant un temps, en plus du fait que les travaux vont coûter cher.

Pour que cet investissement soit pérenne, il faut s'assurer que l'outil retenu est géré comme un commun. Devenir simplement utilisateur, **consommateur** pour mieux dire, d'un logiciel libre, c'est une posture de parasite : il existe, grâce à d'autres, un commun numérique, je profite de ce commun pour gagner en résilience et en autonomie, mais je n'y contribue pas. Pour assurer que l'outil est un commun numérique, et qu'il le reste, il est nécessaire qu'une partie significative des utilisateurs soient également des contributeurs. Et le meilleur moyen d'obtenir cet effet, c'est de devenir soi-même contributeur sur certains logiciels.

Le choix des logiciels auxquels on contribue devient alors assez similaire à un choix d'investissement. Sachant qu'on dispose d'une capacité d'investissement donnée, qui n'est pas infinie, quels sont les projets sur lesquels il est le plus pertinent de faire cet investissement ?

Pour assurer que l'outil est un commun numérique, et qu'il le reste, il est nécessaire qu'une partie significative des utilisateurs soient également des contributeurs.



Ça peut être les logiciels sur lesquels on a une compétence particulière qui fait que l'implication de l'entreprise sera particulièrement significative. C'est par exemple le cas des "logiciels métiers". Pour entretenir les logiciels qui servent aux échanges interbancaires, qui de mieux placé que les grandes banques systémiques ? C'est selon nous une première famille de communs numériques où la valeur de l'investissement est évidente : les logiciels pour lesquels l'entreprise présente une expertise métier incontestable. C'est typiquement un cas où la structure financière pourra être celle d'un syndicat professionnel, ou d'un consortium sectoriel, visant à la normalisation des outils. Par exemple, il semblerait logique que des bibliothèques logicielles gérant les formats d'échange de SEPA dans les différents langages habituels soient écrites et maintenues par les banques centrales européennes, et/ou les grandes banques systémiques de la zone Euro, sous la forme d'un commun numérique utilisable par tous les acteurs de l'économie qui en ont besoin.

Ça peut être les logiciels pour lesquels l'entreprise dispose, en interne, d'expertise technique particulière. Par exemple, parce qu'on a développé un usage très avancé du produit, qui amène à le faire évoluer, à ajouter des fonctionnalités nouvelles. Il devient alors naturel de poursuivre l'implication en ayant un pôle d'expertise qui aide les autres utilisateurs (y compris les concurrents) à s'approprier les nouvelles fonctionnalités, qui contribue à leur maintenance et à leur amélioration, etc.

Et puis, il y a une troisième piste qui semble intéressante, mais qui est plus complexe à détecter. C'est celle des logiciels absolument critiques, mais très peu maintenus, parce que trop centraux. L'exemple le plus évident de ce phénomène, ce sont les bibliothèques de chiffrement (l'exemple de libssl vient en tête). C'est une brique logicielle essentielle, qui est au centre de la sécurisation d'un nombre important de systèmes informatiques, mais qui a longtemps été maintenue par un développeur, tout seul, mal financé, peu soutenu. Des briques essentielles comme celle-là, qui sont presque orphelines, il y en a beaucoup. Elles n'attirent pas le regard : ça fonctionne, c'est maintenu,

Pour entretenir les logiciels qui servent aux échanges interbancaires, qui de mieux placé que les grandes banques systémiques ?



il y a régulièrement des mises à jour, il n'y a pas moyen de faire du business avec ça. Et pourtant, il est vital que ces systèmes soient développés et protégés. Il serait par exemple dramatique qu'une personne malveillante vienne s'impliquer dans la maintenance de ce type de système. Ce serait un moyen très efficace d'introduire des failles de sécurité partout. Cela a déjà été le cas par exemple par la découverte accidentelle d'une porte dérobée dans la suite d'outils XZ, témoignant d'une activité malveillante importante et d'une planification exemplaire de la part d'un État (cf. pages suivantes).

Il y a cependant ici une difficulté : la contribution à un commun numérique ne rentre pas dans les modèles financiers actuels. On peut facilement expliquer que la DSI va dépenser des sommes considérables pour acheter des licences d'un logiciel. Mais on peut difficilement expliquer que la DSI va dépenser les mêmes sommes pour contribuer à ajouter des fonctionnalités dans un logiciel qui ne sont pas les fonctionnalités dont l'entreprise a immédiatement besoin, mais celles dont la communauté des utilisateurs a besoin.





Attaque de XZ Utils

(source: [Wikipedia](#))

Selon Wikipedia : « L'attaque de XZ Utils par porte dérobée est une grave atteinte à l'intégrité de son code ayant visé par rebond le serveur OpenSSH qui a été détectée fin mars 2024 par le développeur allemand Andres Freund. Selon de nombreux analystes de sécurité, elle constitue une attaque de la chaîne logistique (en) de Linux d'une envergure encore inégalée. »

Le principe de l'attaque est très subtil. La bibliothèque XZ Utils est un outil de compression très efficace et repris par de nombreux logiciels, soit pour (dé)compresser des données sur le disque, soit pour compresser les données lors d'un transfert. L'attaque visait l'utilisation de XZ faite par OpenSSH, pour y introduire une porte dérobée.

L'élément central pour notre propos ici n'est pas la méthode technique, mais le fonctionnement humain et organisationnel.

La bibliothèque XZ Utils était maintenue essentiellement par une seule personne, ce qui fait qu'un deuxième contributeur a été rapidement accepté et qu'il a pu rapidement gagner des droits de co-mainteneur sur le projet.

C'est un petit projet (par rapport à d'autres comme le noyau Linux ou PostgreSQL), relativement mature et stable, largement adopté et utilisé, y compris par des logiciels très sensibles en matière de sécurité. Ce qui rend le projet particulièrement vulnérable, c'est l'absence d'une communauté autour, qui fait que le développeur principal est seul sur le sujet, pendant trop de temps.

C'est l'exemple type de ce que nous désignons comme des projets très critiques, parce que très centraux, mais également ne faisant pas l'objet d'un investissement des industriels, parce que ce n'est pas une source intéressante de business. Si une petite communauté, de 5-10 co-mainteneurs du projet, venant tous de groupes industriels différents, avait existé, il est probable que l'introduction de code malveillant aurait été détectée plus efficacement.

Il y a là une difficulté organisationnelle : intégrer que le suivi au long cours d'un tel projet, par un développeur isolé mais compétent dans l'entreprise, est une contribution importante à un commun numérique stratégique. Et donc financer ce développeur.



Ce qui rend le projet particulièrement vulnérable, c'est l'absence d'une communauté autour.



05

CHANGER DE CULTURE

Changement de culture

S'appuyer sur des communs pour assurer la résilience est un changement culturel important, qui risque de prendre très longtemps. Il ne faut pas le considérer comme un sujet spécifique au numérique, ou à la question de la résilience face aux dangers de la dépendance technologique.

C'est le même problème conceptuel que le rapport à l'environnement : il y a un **intérêt de court terme** à s'accaparer les ressources, et à les exploiter au-delà de ce que l'environnement peut produire, mais il y a un **intérêt de long terme** à ne pas le faire.

De la même manière, chacun a un intérêt de court terme à utiliser le logiciel libre, et chacun a un **intérêt individuel de court terme à ne pas contribuer** (c'est moins cher). En revanche, tout le monde a un **intérêt commun, de long terme, à contribuer** pour que le commun numérique reste disponible à tous.

À cet endroit-là, on retrouve la notion de commun : il faut contribuer, entretenir la ressource commune, y compris en modérant ses propres appétits et en luttant contre son intérêt de court terme. Et cette notion n'est pas naturelle du tout dans nos organisations, elle ne correspond pas à une minimisation des coûts, elle correspond à une maximisation des gains par la coopération².

S'appuyer sur des communs pour assurer la résilience est un changement culturel important, qui risque de prendre très longtemps.

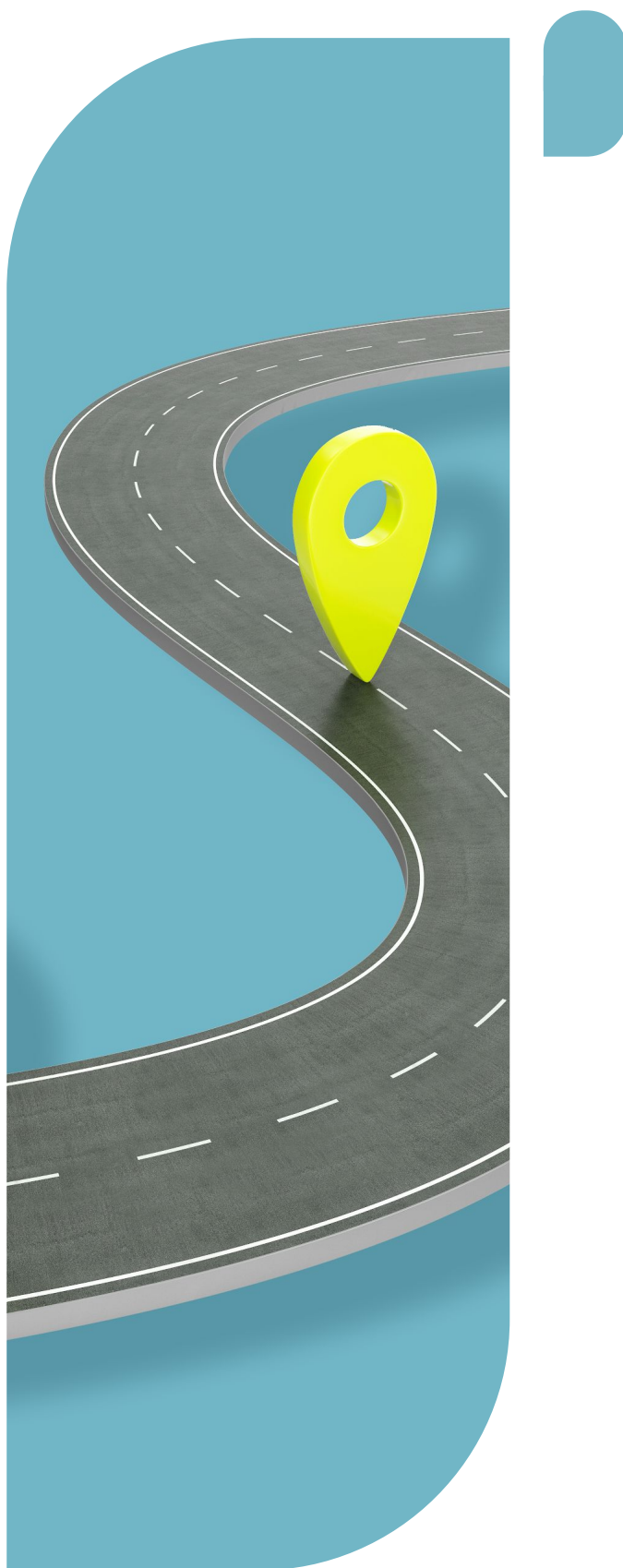


² On retrouve ici la notion d'équilibre de Nash. En théorie de la régulation économique, on parle d'équilibre de Nash pour une situation où tous les acteurs auraient un intérêt collectif à changer une stratégie, mais aucun n'y a un intérêt de court terme évident. C'est souvent cité comme un des exemples de cas où il faut un régulateur. Aucun acteur ne fera le choix du changement, puisque c'est contre son intérêt. Ce qui serait dans son intérêt, c'est que tout le monde le fasse, y compris lui. Mais il ne peut pas forcer ses concurrents. Si un régulateur impose le changement, alors cela devient possible.

Plusieurs difficultés apparaissent :

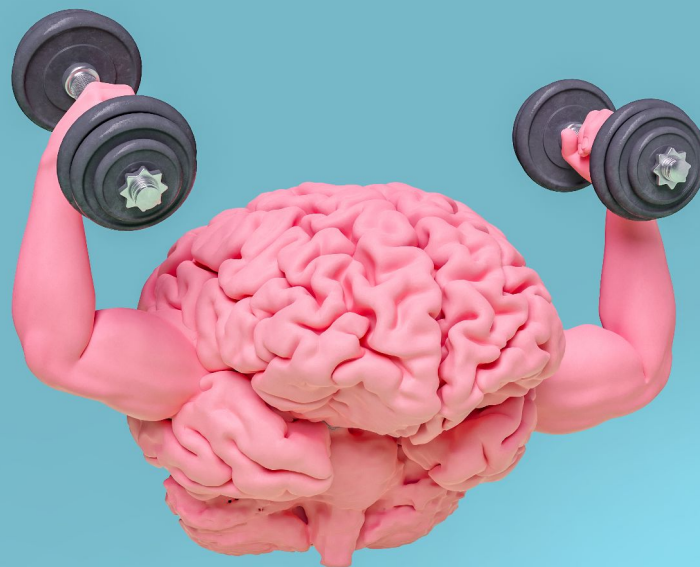
- il existe un intérêt de court terme à ne pas contribuer ;
- il n'existe qu'un intérêt de long terme à contribuer ;
- donc, il y a un risque d'arbitrage en défaveur de la contribution au commun à chaque soubresaut dans la vie de l'entreprise ;
- pire, la contribution va améliorer le produit, également utilisé par les concurrents, qui sauront peut-être mieux que nous tirer partie des améliorations futures.

On retrouve cette question sous beaucoup de formes : comment valoriser dans la comptabilité les investissements visant à protéger / développer le commun ? Pour l'environnement, on a utilisé la sanction, qui permet de faire rentrer dans la mesure en Euros le risque à ne pas faire (risque de sanction à mettre en face du coût de l'action attendue).



Un autre changement culturel complexe est de se défaire d'un complexe d'infériorité beaucoup trop intériorisé. Il se manifeste par un manque de confiance dans les solutions logicielles développées en France ou en Europe. C'est l'idée que le nouveau logiciel fait par les américains est merveilleux, alors que les français ou les européens vont faire beaucoup moins bien. C'est en général faux, en moyenne. Mais ce complexe d'infériorité est tellement intégré qu'il devient difficile de s'en défaire. Ainsi, beaucoup pensent, dans le monde de l'industrie, que les solutions développées en dehors des USA en matière de numérique, de cloud, ou d'IA, sont bien plus faibles. Or, en général, ce n'est pas le cas. La recherche est au bon niveau, les outils quand ils existent sont au bon niveau. Ce qui est courant, c'est ce déficit d'image, qui fait qu'il y a un déficit de commandes, donc moins de moyens à disposition pour améliorer les produits, et une défiance des investisseurs, puisqu'il n'y a pas un bon volume de commandes. Mais ce n'est pas par manque de savoir faire, ou par manque de qualité des résultats.

Quand les américains viennent dire qu'ils sont les leaders de l'IA et qu'ils vont le rester (par exemple, le discours de JD Vance sur le sujet), on est tentés en Europe de les croire, précisément, parce que ce complexe d'infériorité a été complètement intégré.





06

DÉPLACER LE FINANCEMENT

Déplacer le financement

D'un point de vue strictement financier, le fait que l'industrie tende à remplacer des solutions fermées par des solutions basées sur des communs numériques, c'est une simple modification du canal de financement du développement des briques logicielles communes.

Dans le modèle avec un logiciel fermé, il y a un intérêt à payer très cher la licence d'utilisation du logiciel. D'une part, c'est ce qui permet d'assurer que l'éditeur aura les moyens de développer le produit que nous utilisons. Si l'éditeur est bien financé on améliore les perspectives d'amélioration du produit. D'autre part, ce coût de licence élevé est un facteur d'éviction pour nos concurrents, en particulier pour les nouveaux entrants sur le marché qui n'ont pas les moyens de payer aussi cher.

Dans le modèle avec un commun numérique, il faut aussi financer régulièrement (par exemple en mettant à disposition des ingénieurs), mais le rôle d'éviction disparaît. Le logiciel est disponible pour tous, y compris pour les nouveaux entrants sur le marché. L'efficacité économique est plus grande, parce que les efforts de développement sont mis en commun (pour un logiciel propriétaire, cet effet de mutualisation du développement n'a lieu que quand le produit est en situation de monopole, créant un risque immense sur la résilience), potentiellement sur une structure qui peut être sans but lucratif (donc économiquement plus efficace).

Ce n'est donc pas structurellement un choix plus coûteux dans le modèle de développement, et ça ne remet pas en cause l'ensemble de l'activité économique de l'entreprise. Mais cela suppose, et c'est toute la difficulté, un modèle qui favorise la coopération. La principale difficulté est donc d'ordre culturel et non économique ou technique.

Dans le modèle avec un logiciel fermé, il y a un intérêt à payer très cher la licence d'utilisation du logiciel.

Dans le modèle avec un commun numérique, il faut aussi financer régulièrement.





07

MISE EN PRATIQUE



Et concrètement, ça donne quoi ?

Pour donner un peu de corps à cet exposé, regardons ensemble quelques exemples. Pas une liste exhaustive bien entendu, mais des types de problèmes, et des types de solutions envisageables pour traiter la difficulté



La bureautique en ligne

Le sujet est simple. Comment traiter la bureautique en ligne, la messagerie et tous les outils connexes, autrement qu'en devenant consommateurs passifs des solutions des géants américains (la suite Google d'un côté, la suite Microsoft de l'autre, pour faire simple) ? Plusieurs solutions sont à disposition.

D'une part, il existe des prestataires de services similaires, en France ou en Europe. Bien entendu, ce n'est pas la même solution, et on est toujours un peu dépaycé. Parfois, ce sont des solutions moins complètes, mais du mail et de la bureautique simples, il y a beaucoup de monde capable de le fournir. La question devient alors : si c'est un enjeu stratégique, une dépendance qui pose problème (par exemple parce qu'en cas de soubresaut géopolitique, mon organisation peut être paralysée durablement parce que le mail a disparu), le recours à un prestataire local est tout à fait jouable.

Pour beaucoup d'organisations, il est même possible de construire de telles suites logicielles. Pour cette raison que toutes les briques sont disponibles, en logiciel libre le plus souvent. L'effort de configuration, d'installation, d'intégration entre les différentes briques est un projet d'envergure, en soi. Devoir refaire l'intégration de tous ces outils, pour une PME qui aurait 50 comptes à gérer, ça n'a aucun sens. Mais travailler sur l'intégration de ces outils, quand on est une organisation de grande envergure, c'est un objectif atteignable. Pour le dire crument, c'est un projet en millions, pas un projet en milliards.

Et par ailleurs, puisque beaucoup de briques existent sous la forme d'un commun numérique, il est tout à fait raisonnable de contribuer à un nouveau commun, soit pour les briques manquantes, soit pour les solutions d'intégration. C'est par exemple l'ambition des projets autour de la *Suite numérique* de la direction interministérielle du numérique (DINUM) : mettre en place des outils de travail collaboratif, pour certains agents de l'État, contribuer au développement des produits en proposant des améliorations, mettre à disposition les versions intégrées, soit pour les organisations qui veulent les utiliser en interne, soit pour des prestataires qui voudraient fabriquer une offre à partir de tout ça.

On voit donc deux axes. D'une part, s'intégrer dans la communauté autour d'une brique logicielle. D'autre part, créer une nouvelle brique logicielle (l'intégration de l'ensemble en un produit cohérent). Il ne manque plus que la notion de travail communautaire pour que cette nouvelle brique devienne un commun numérique. Or cet objectif est atteignable : que des grands groupes décident d'utiliser en interne ces solutions intégrées, que des PME se saisissent des produits pour en faire des offres (par exemple à destination des TPE, ou de certaines collectivités) et que ces acteurs contribuent à faire progresser ce produit commun, et on aura fabriqué un commun numérique solide, disponible pour tous, qui aura permis de réduire une dépendance stratégique dangereuse.

La fausse pensée stratégique « **Nous allons nous concentrer sur notre core business** » est ici une bêtise. Si mon entreprise a un besoin vital d'avoir du mail pour pouvoir fonctionner, c'est un besoin essentiel. Si l'écosystème existant n'est pas assez résilient et diversifié, alors ça devient un enjeu de **core business** que de contribuer à aplanir ces difficultés. Surtout si ça ne se traduit pas par un surcoût trop élevé.

La fausse pensée stratégique

« **Nous allons nous concentrer sur notre core business** » est ici une bêtise.



Stratégie de sortie de VMware

Le sujet que nous allons regarder ici est celui de la dépendance à la solution **vSphere** de **VMware**. Il est apparu au grand jour à la suite du rachat de **VMware** par **Broadcom**, et du changement de modèle de licence, et de tarif, qui en a découlé.

VMware est le leader des solutions de virtualisation. C'est une brique essentielle dans l'hébergement des systèmes d'information sur les infrastructures internes, celle qui permet de gérer des machines virtuelles (VM) sur les machines physiques.

De nombreuses entreprises ont choisi de rationaliser leurs solutions de virtualisation autour de la seule et unique solution **VMware** pour tous leurs systèmes. Celle-ci est devenue au fil du temps en situation de quasi monopole sur ce secteur d'activité, la quasi-totalité des grands acteurs économiques l'exploitent pour faire fonctionner leurs systèmes d'information.

La modification du modèle de licence et du prix³ a mis en évidence le problème : une dépendance forte à un fournisseur unique en situation de quasi monopole, sur un modèle technique non-normalisé, et donc sans remplaçant immédiatement disponible sur étagère.

Plusieurs pistes s'offrent aux entreprises qui souhaitent se défaire de cette dépendance. Nous allons examiner deux grands types d'approches.

La première approche est de trouver une alternative à **vSphere**. L'écosystème actuel est riche de concurrents et la migration de machines virtuelles d'un système à l'autre est aujourd'hui considérée comme normalisée. Le changement de l'enveloppe de virtualisation n'est donc pas le point de dépendance principal. Les défis se situent dans la complexité d'intégration et d'opération de la plateforme d'hébergement. Toute la chaîne technique est touchée : la CI/CD, les briques d'**Infrastructure as Code** (IaC), la supervision, les systèmes de réplication, de sécurité, de segmentation du réseau, l'intégration dans les chaînes de provisioning des ressources, etc.

³ On parle de quelque chose comme 500% d'augmentation, sur des montants parfois en millions d'euros, par le passage d'un modèle d'achat de licences à un modèle d'abonnement, bouleversant complètement le modèle économique.

Repartir sur une solution concurrente (par exemple Nutanix) ne pose pas de difficulté pour l'hébergement des machines virtuelles, mais les conséquences pour refaire l'intégration de tous les systèmes sont lourdes. Pour reprendre une métaphore précédente : on a déplacé la chaufferie au troisième étage, tout est à revoir dans le bâtiment. Quitte à choisir cette approche, il vaut mieux s'appuyer sur des systèmes qui ne présentent pas le même risque lié au monopole (par exemple des systèmes de virtualisation qui s'appuient sur des logiciels libres ayant un développement communautaire comme Qemu ou KVM), mais ces solutions sont en général moins packagées, et encore plus complexes à intégrer dans la chaîne de développement.

La seconde approche est de profiter de cette contrainte pour changer de paradigme technique. En effet, le modèle de développement qui s'appuie sur des machines virtuelles est un peu daté. Les architectures modernes sont conçues avec des conteneurs, des micro-services, et une séparation forte entre le code et les données. Ces nouvelles approches permettent la construction de services plus modulaires, plus granulaires facilitant leur réutilisation et leur portabilité.

Par conséquent, aujourd'hui les machines virtuelles ne sont plus perçues comme la cible de déploiement par défaut des applications modernes. On préfère de nos jours des systèmes d'orchestration de conteneurs, le standard actuel pour ça étant Kubernetes. Il a par ailleurs été adopté par tous les acteurs du cloud, y compris les hyperscalers. Mais Kubernetes permet également d'être déployé sur des serveurs physiques, et donc de s'affranchir de la dépendance aux hyperviseurs comme vSphere.

Ces nouvelles approches permettent la construction de services plus modulaires, plus granulaires facilitant leur réutilisation et leur portabilité.



Il se trouve, en plus, que **Kubernetes** est un logiciel libre, qui s'appuie sur des briques logicielles essentiellement maintenues par une communauté, très active. C'est donc une solution qui permet à la fois de moderniser le système d'information dans son architecture, de continuer à opérer en propre les systèmes les plus critiques, de s'affranchir de la dépendance à une technologie propriétaire devenue trop centrale, et de passer sur un modèle qui permettra, si on le souhaite, de déplacer certains systèmes chez un fournisseur de cloud.

En revanche, cette nouvelle cible nécessite de transformer en profondeur ses services au niveau applicatif et pas uniquement sur l'enveloppe de déploiement. C'est aussi compliqué, et peut-être même plus, que le simple remplacement de **VMware** par un concurrent. Mais, si cette modernisation des systèmes est déjà dans les objectifs de la DSI, alors c'est une bonne opportunité pour commencer.



Cette nouvelle cible nécessite de transformer en profondeur ses services au niveau applicatif et pas uniquement sur l'enveloppe de déploiement.

Le cloud et les hyperscalers

Le sujet que nous allons effleurer ici est celui de la dépendance aux **hyperscalers** en matière de **cloud computing**. Que ce soient les géants américains (**AWS**, **GCP**, **Azure**) ou le géant chinois (**Alibaba**), le sujet est le même.

Il y a ici une dépendance très forte. Quand des pans entiers d'un système d'information sont migrés sur des solutions de cloud, le fournisseur retenu a la capacité de faire disparaître le système d'information de l'entreprise, du jour au lendemain. Ou, en reprenant l'exemple de ce qui est arrivé avec **VMware**, la grille tarifaire peut évoluer dans une direction défavorable.

La première piste qui vient à l'esprit est de travailler en multi-cloud. Dans la version idéale, chacun des outils qu'on migre vers ces solutions est traité pour pouvoir être déployé sur les systèmes de différents fournisseurs, ainsi on pourra automatiquement déplacer ces applications d'un fournisseur vers un autre en cas de difficulté. Cette solution, séduisante sur le papier, est plutôt illusoire. En effet, les interfaces de pilotage de ces fournisseurs (typiquement, le provider **Terraform**, ou l'API) sont fondamentalement incompatibles entre elles. Et cela revient donc, peu ou prou, à faire plusieurs fois le travail de migration. Par exemple à produire une version de l'infra-as-code (IaC) compatible avec **AWS**, une autre compatible avec **Scaleway** et une troisième compatible avec un **ProxMox** hébergé en interne. Et à s'assurer que ces trois versions sont en permanence valides. Et à s'assurer en permanence qu'on sait basculer de l'une à l'autre dans de bonnes conditions. Un enfer de complexité. Ça peut être une solution au problème, mais il faut en assumer le coût et la complexité, et la réserver aux cas où c'est la solution la mieux adaptée.

Une difficulté particulière, en plus des interfaces qui diffèrent, est que toutes les fonctionnalités ne sont pas disponibles partout. Si notre application s'appuie sur une particularité du fournisseur A (mettons, un point précis des spécifications des security groups sur cette plateforme), et que les autres fournisseurs retenus n'apportent pas cette spécificité, alors il faudra faire des design très différents. C'est aussi le cas si on s'appuie sur une offre disponible à un endroit et pas à l'autre. Mettons, un service de **PostgreSql** managé d'un côté et de **MySql** managé de l'autre. Soit il faut reprendre tout le développement de l'application, pour qu'elle soit compatible avec les deux systèmes, soit il faut créer un **PostgreSql** managé sur la plateforme qui ne le propose pas, et le gérer nous même.

On est ici au cœur d'une dépendance technologique à un fournisseur. À chaque fois qu'on fait appel, même sans en être conscient, à une particularité du système, on rend plus complexe l'éventuelle migration hors de ce système, et on augmente donc le coût de cette dépendance, sans le savoir.

Le simple fait de limiter les fonctionnalités sur lesquelles on s'appuie à des fonctionnalités très génériques, plutôt que de chercher à exploiter au plus près les finesses du modèle proposé, est un outil, dans la gestion des projets, qui permet de diminuer le coût de sortie du fournisseur qu'on a retenu, et donc un bon moyen de réduire la dépendance.

Une autre approche peut être de déplacer la couche technique où se trouve la dépendance. Ainsi, pour ne pas dépendre d'un fournisseur unique (mettons, AWS) on peut choisir de dépendre d'un modèle générique (par exemple **Openstack**) disponible chez plusieurs fournisseurs. Le passage d'un fournisseur à l'autre ne va pas devenir magiquement évident, mais il en sera tout de même grandement simplifié. Si notre application utilise **Openstack** tel qu'il existe chez OVH, le passage sur un **Openstack** hébergé en interne, ou hébergé par un autre fournisseur, sera plus simple que de tout réécrire.

C'est d'ailleurs en partie cette analyse qui a poussé autant d'entreprises vers **VMware** : ces entreprises ont fait le choix d'avoir une infrastructure virtualisée, mais souvent sur des équipements détenus en propre, et **VMware** semblait une solution technique très adaptée. C'est un cas où on a déplacé la dépendance, d'un fournisseur de cloud vers un éditeur de solution logicielle, sans se préoccuper du fait que ce n'est pas un commun numérique.

Le passage d'un fournisseur à l'autre ne va pas devenir magiquement évident, mais il en sera tout de même grandement simplifié.



Tout l'équilibre ici est de comprendre où sont les dépendances, d'être capable de dire si elles sont acceptables ou non, et de traiter la difficulté qui en découle.

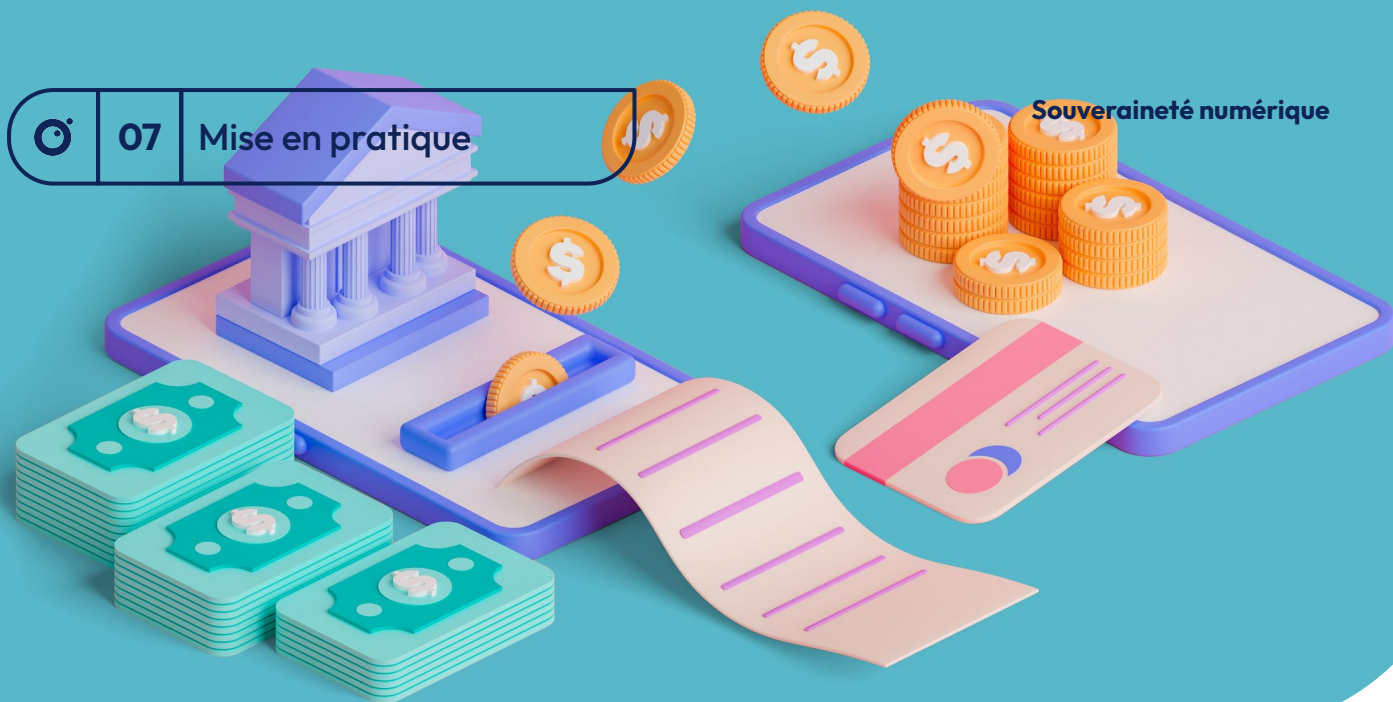
Par exemple, si la dépendance à un fournisseur américain ou chinois pose un problème géopolitique qui n'est pas acceptable, alors le simple fait de faire appel à un fournisseur européen, quitte à ce que ce soit avec une offre moins riche, résout le problème.

Par exemple, si la dépendance à un fournisseur unique pose un problème de dépendance économique, le fait de changer de fournisseur ne résout rien (passer du cloud AWS à un VMware sur les infrastructures internes ne change rien). En revanche, le fait de changer de technologie pour choisir une technologie disponible chez différents fournisseurs résout le problème.

Par exemple, si la dépendance à un éditeur d'une solution propriétaire est une dépendance qui n'est pas acceptable (les cas VMware ou Oracle dont on a beaucoup parlé), alors le recours à un commun numérique, quel que soit le loueur d'infrastructure qu'on utilisera pour l'héberger, est une façon de traiter la difficulté.



Pour comprendre le type de solution dont on a besoin, il faut avant tout comprendre la nature du problème qu'on souhaite traiter.



Les logiciels métiers

Le cas que nous voulons examiner ici est celui des logiciels “métier”, qui permettent typiquement de répondre à une norme qui n’a de sens que dans un métier précis, mais qui s’applique à tous les acteurs du métier en question.

Un premier exemple est celui de la facturation dématérialisée. Il existe une norme, mise en place par la puissance publique. Cette norme est ré-implémentée par chaque éditeur. Cette norme offre un intérêt industriel pour les entreprises qui en sont utilisatrices (elles peuvent passer d’un prestataire à l’autre). Mais le fait de refaire entièrement l’implémentation pour chacun des éditeurs est un choix discutable. On aurait pu envisager qu’une bibliothèque de code commune soit développée pour mutualiser le coût de développement associé.

Un deuxième exemple, très différent dans le principe, est le mode de fonctionnement des normes du réseau. Le plus souvent, ces normes sont écrites par des comités qui regroupent des salariés de différentes entités, entreprises, universités, laboratoires, éditeurs, équipementiers, qui ont un intérêt et une expertise sur la norme en cours d’établissement. Le plus souvent, la norme technique est accompagnée d’un bout de code qui la réalise, ou qui la réalise partiellement, parce que le bout de code est toujours moins ambigu qu’un texte de normalisation, même bien écrit. En général, le bout de code adjoint à la norme n’est pas intégré tel quel, mais il servira de base pour les développements des différents éditeurs. On a là une approche qui permet de s’assurer que la norme est réaliste (c’est le but), et que tout le monde pourra l’implémenter rapidement avec un coût de développement raisonnable.

Un troisième exemple est celui des normes interbancaires. En France, historiquement, c’est le CFONB (Comité Français d’organisation et de normalisation bancaire) qui se chargeait de définir ces normes communes.

C'est par exemple ce comité qui définissait les formats de fichier échangés par les vieux programmes en COBOL (les fichiers ETEBAC que s'échangeaient les banques entre elles et les entreprises avec les banques, pour faire des virements en masse ou les prélèvements automatiques). C'est dans les mêmes sphères que se discutent les normes actuelles, autour de l'Euro (par exemple, les normes SEPA pour les prélèvements et les virements dans la zone Euro). On pourrait imaginer que ces normes existent également sous la forme d'un commun numérique, par exemple des bibliothèques logicielles disponibles dans les langages les plus usuels, et maintenues par les équipes des comités de normalisation. Ou sous la forme d'outils de test (typiquement, des outils permettant la validation et la certification d'une solution, qui pourront être intégrés dans la chaîne CI/CD d'un éditeur).

Ces exemples ont tous un point commun. Il est question de mettre en place une norme, partagée entre concurrents, qui ne représente pas un avantage compétitif, mais qui correspond à un besoin commun. Ce n'est pas la qualité de l'implémentation de la norme SEPA (absence de bug, performance du code, etc) qui fait qu'on choisit une banque ou une autre. Ce n'est pas un avantage compétitif. Mais c'est une nécessité, et cette nécessité est commune à tous les acteurs, aussi bien les banques que leurs clients. Ré-implémenter, avec tout le cortège d'erreurs et d'anomalies, ces normes à chaque fois est une perte d'efficacité. L'existence d'un lieu de coopération où ces normes peuvent se traduire en un commun numérique serait un apport, en résilience de l'ensemble du système économique concerné.

Bien entendu, ce n'est pas ici que va se situer l'innovation. Si on reprend l'exemple du réseau, les fonctionnalités normalisées (mettons, le VLAN-tagging) ne sont plus un différenciant entre les différents fournisseurs d'équipements (tout le monde a la fonctionnalité, et ça fait un moment que ça marche chez tout le monde et que c'est interopérable). Les fonctionnalités qui sont des différenciants majeurs sont celles qui ne sont pas encore normalisables, parce que chaque équipementier invente sa solution pour traiter le sujet. C'est à la fin, quand une solution aura pris le pas sur les autres, ou quand les acteurs décideront de travailler ensemble à définir une norme commune qui reprend les avantages de chacune de leurs solutions spécifiques qu'une nouvelle norme va apparaître. Et c'est à ce moment-là qu'il y a un intérêt à produire un commun numérique.



CONCLUSION



| CONCLUSION |

On l'a vu, le sujet du cloud souverain n'est pas tellement une question de souveraineté, c'est surtout une question de comprendre et de gérer les risques liés à la dépendance vis-à-vis d'une technologie ou d'un fournisseur. La notion d'autonomie stratégique, avec comme objectif la résilience, est une meilleure description du sujet.

Tout l'enjeu est d'identifier les endroits stratégiques où une dépendance inacceptable est en place, ou risque de s'installer, et de traiter le problème. Pour cela, la meilleure des solutions est d'avoir recours aux communs numériques, parce qu'ils garantissent une solution stable, pérenne, et indépendante, parce que dépendante de tous, et non de quelques acteurs en position de force.

Il y a donc un besoin de pivoter, de changer de culture. D'une part, pour rentrer dans les modèles d'une économie des biens non-rivaux, dans laquelle on ne paye pas un contrat de licence à un éditeur, mais on finance une contribution aux communs pour pouvoir en profiter sans être un parasite du système. D'autre part, il faut accepter de sortir de notre complexe d'infériorité sur tout ce qui touche au numérique. Ce sont deux changements culturels profonds.

Le sujet, c'est d'accepter de faire face aux responsabilités techniques, plutôt que de s'en remettre passivement à un fournisseur qu'on a choisi parce que c'est le plus gros du marché et que donc on ne nous reprochera jamais de l'avoir choisi.

Vouloir être souverain sur ses choix, c'est bien au départ les assumer, et ensuite chercher à construire un environnement qui permette d'évoluer avec indépendance. Cela peut se traduire, parfois, par un arbitrage contre la performance et l'optimisation, mais en faveur de la robustesse et de la résilience.



Les pistes que nous proposons ici permettent de traiter cette question de dépendance technique :

Contribuer à l'émergence de normes.



Utiliser les communs numériques quand ils sont disponibles, y compris si on reste chez les fournisseurs habituels.



Contribuer à l'entretien de ces communs numériques quand on peut

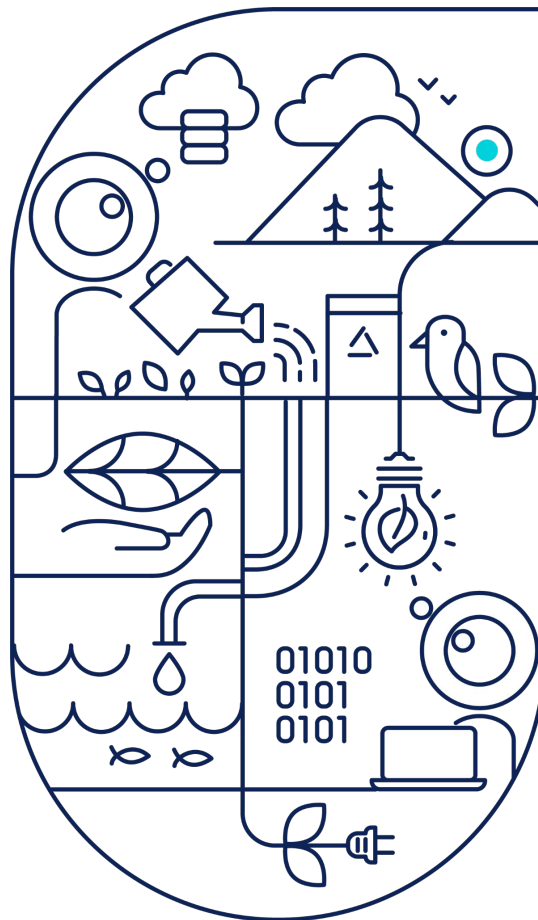


Éviter les technologies non-normalisées qui vont avoir un effet d'emprisonnement quand elles ne sont pas issues d'un commun numérique.





NOTRE WHY OCTO.





NOTRE WHY OCTO.

Dans un **MONDE** complexe aux **RESSOURCES** finies, nous recherchons ensemble de meilleures façons d'**AGIR**. Nous **ŒUVRONS** à concevoir et à réaliser les produits numériques **ESSENTIELS** au **PROGRÈS** de nos clients et à l'émergence d'écosystèmes **VERTUEUX**.



CRÉÉE EN 1998



autre cercle



IMPLANTATIONS

Paris
Toulouse
Lille

FORMATION

octo academy
LEARN TO CHANGE



7 PARTICIPATIONS
OCTO DEPUIS 2011



4 CONFÉRENCES



La Conf Data I IA by OCTO



La conférence tech par OCTO



La conférence Produit par OCTO

4 EXPERTISES & UNE DIRECTION
SCIENTIFIQUE DES LIVRES BLANCS



DATA



SOFTWARE



PRODUIT



CLOUD



**THERE IS
A BETTER
WAY.**